In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [11]:
```
# %pip install opencv-python
```

In [ ]:
```
# Write a program to solve a 0-1 Knapsack problem using dynamic programming
# or branch and bound strategy.
# 02
# 5)
# Design n-Queens matrix having first Queen placed. Use backtracking to plac
# remaining Queens to generate the final n-queen's matrix. 01
# 6)
# Write a program to implement dictionary using hashing. (Use separate
# # chaining)
```

In [15]:
```
def knapsack(weights, profits, capacity):
    dp = [0] * (capacity+1)

    for i in range(len(weights)):
        weight = weights[i]
        profit = profits[i]

        for w in range(capacity, weight - 1, -1):
            take = profit + dp[w-weight]
            skip = dp[w]

            dp[w] = max(take,skip)
    return dp[capacity]

n = int(input("enter the no. of items: "))


weights = []
profits = []

for i in range(n):

    weight = int(input(f"Enter the item weight{i+1}: "))
    profit = int(input(f"Enter the item profit{i+1}: "))

    weights.append(weight)
    profits.append(profit)

capacity = int(input(f"Enter the capacity: "))

answer = knapsack(weights, profits, capacity)

print("Maximum Profit: ", answer)
```

```
Maximum Profit:  91
```

In [ ]:
```

```